

RESEARCH ARTICLE

Flexible Job Shop Scheduling Method for Optimizing Mold Resource Setup Time

YI CHENG¹, ZHIJUN XIE¹, YU XIN¹, KEWEI CHEN², AND ROOZBEH ZAREI³¹Faculty of Electrical Engineering and Computer Science, Ningbo University, Ningbo 315211, China²Faculty of Mechanical Engineering, Ningbo University, Ningbo 315211, China³School of Information Technology, Deakin University, Melbourne, VIC 3125, Australia

Corresponding author: Zhijun Xie (xiezhijun3243@outlook.com)

This work was supported in part by the National Natural Science Foundation of China under Grant U20A20121; in part by Ningbo Public Welfare Project 202002N3109 and Project 2022S094; in part by the Natural Science Foundation of Zhejiang Province under Grant LY21F020006; in part by the International Cooperation Project of Ningbo under Grant 2023H012 and Grant 2023H007; in part by the Science and Technology Innovation 2025 Major Project of Ningbo under Grant 2019B10125, Grant 2019B10028, Grant 2020Z016, Grant 2021Z031, Grant 2022Z074, Grant 2022Z241, Grant 2023Z132, Grant 2023Z133, and Grant 2023Z216; and in part by Ningbo Fenghua District Industrial Chain Key Core Technology “Unveiled the Commander” Project 202106206.

ABSTRACT In a practical development environment, completing a job often necessitates the coordinated allocation of multiple resources. However, resource constraints such as molds are rarely considered in the multi-resource job shop scheduling problem. These molds have sequence-dependent setup times and are fully occupied during processing. Therefore, this study focuses on the Flexible Job Shop Scheduling Problem with Mold Resource Setup Time Optimization (FJSP-MRST), which aims to minimize setup time and makespan. Considering the characteristics of mold setup time, this paper introduces a Mold-Machine Double Fusion Insertion Method to optimize the mold-machine usage order and improve the mold change time calculation method. This innovative approach improves the overall scheduling process by efficiently managing the utilization of molds and machines. Furthermore, we propose a Mold-Operation Layer Loading Method to compress setup time, offering a novel strategy for reducing the setup time for mold resources. These two pioneering methods are integrated with an enhanced multi-objective differential evolution algorithm to obtain the optimal solution for FJSP-MRST. Experimental results demonstrate that the proposed methods can render the arrangement of machines and molds under the current process sequence more reasonable and effectively eliminate implicit setup time.

INDEX TERMS Flexible job shop scheduling problem, mold resource, setup time, differential evolution algorithm.

I. INTRODUCTION

In practical workshop settings, the Flexible Job Shop Scheduling Problem (FJSP) often necessitates the consideration of not only machine selection and process sequencing but also the allocation of resources beyond machines [1]. However, when examining the issue of multi-resources, most scholars focus on the constraints related to machines and human resources [2], while neglecting resource constraints such as molds that have sequence-dependent setup times [3] and are fully occupied during machining. This particular type

of resource constraint leads to extended setup times and idle waiting periods during the machining process in workshop scheduling.

This resource constraint is prevalent [4] in many manufacturing companies [5]. For instance, in the automotive parts manufacturing industry, the die-casting process for a part involves multiple steps that require the use of a mold to determine the part's shape or a corresponding fixture to secure the part during machining. Both a mold and a fixture are classified as the same type of resource. This resource type possesses two distinct characteristics. Firstly, the assembly of the corresponding resources necessitates a lengthy setup before the process initiation. Additionally,

The associate editor coordinating the review of this manuscript and approving it for publication was Cong Pu¹.

the resource is unique and specific to the entire process, requiring a particular resource for each process that cannot be interchanged arbitrarily.

These characteristics pose significant challenges in solving the FJSP that considers such resources. For example, arranging the operation sequence, machine selection, and mold selection reasonably is complex. The operations of the same job need to be sequenced. Moreover, many operations require the selection of machines and molds corresponding to the operation. Machines and molds are uniquely occupied during machining. An unreasonable arrangement can result in frequent mold and machine changes or long idle waiting periods due to waiting for the completion of the previous process or the occupation of machines and molds. Additionally, numerous stringent constraints are difficult to optimize. The operation, machine, and mold all have specific constraint conditions. As a result, optimizing the overall scheduling solution without violating these constraints is challenging.

The universality of resources and the complexity of the problem have created an urgent need for manufacturing companies to find a solution. They are looking for a more general method to effectively arrange the operation sequence and select machines and resources. Although various models have been developed by scholars to address multi-resource scheduling problems in different production environments [6], [7], [8], and algorithms have been developed to consider setup times [9], [10], there has been no systematic consideration of how to optimize scheduling problems with this particular type of resource. This gap in the literature has inspired our investigation of scheduling problems that feature this characteristic resource.

Optimizing setup time is crucial for seeking improved scheduling solutions for this problem [11], [12]. In this paper, we use mold resources as an example and define the problem as a flexible job shop scheduling problem considering mold resource setup time optimization. Given the strong global search ability and practical application of the multi-objective differential evolution algorithm (MODE) [13], [14] in FJSP, we have enhanced MODE to suit the current problem. Moreover, we have introduced the Mold-Machine Double Fusion Insertion Method to adjust the chromosome, addressing the challenge of arranging machines and molds reasonably. This approach reduces the number of mold changes, eliminates unnecessary setup time, and accelerates the convergence speed of the algorithm. Additionally, we have proposed the Mold-Operation Layer Loading Method, optimizing the loading process and removing hidden setup time by relaxing some stringent constraints between molds and operations. Consequently, the compressible mold-changing process can be carried out in advance. The main contributions of this paper are as follows:

- 1) A Mold-Machine Double Fusion Insertion Method is proposed to enhance chromosome quality. Through related experiments, it has been shown that in 70% of cases, this method can adjust chromosome effects to their optimal level.

- 2) Proposing a Mold-Operation Layer Loading Method that can eliminate hidden mold change times in the vast majority of cases.
- 3) Improvement of the Differential Evolution Algorithm through the combination of the two aforementioned methods and the joint adjustment of evolution chromosomes for process, machine, and mold encoding. Experimental results demonstrate that this algorithm is superior to existing algorithms commonly used for the FJSP.

The rest of this paper is organized as follows. Section II summarizes relevant literature in recent years. Section III describes the flexible job shop scheduling problem with mold resource setup time optimization. Section IV presents the Mold-Machine Double Fusion Insertion and Mold-Operation Layer Loading Methods. An enhanced multi-objective differential evolutionary algorithm that integrates these two techniques is also introduced. Section V reports the case study. Section VI wraps up the paper, offering suggestions for future research directions.

II. RELATED WORKS

Constraints in the flexible multi-resource job shop scheduling problem typically focus on various resources. For example, He et al. [15] explored a dual-resource-constrained flexible job shop scheduling problem, which involved machine and worker constraints. They introduced an improved African vulture optimization algorithm to address this issue, utilizing the AVOA algorithm based on position updating formulas and position vectors. Gong et al. [16], in their work, simultaneously took into account processing time, green production, and worker skills. They established a multi-objective dual-resource-constrained model, which they resolved using a non-dominated sorting genetic algorithm. Wang et al. [17] directed their attention to the energy-saving multi-resource flexible job shop scheduling problem. Liu et al. [18] extensively investigated the application of evolutionary algorithms in addressing the fuzzy flexible job shop problem with uncertain processing times, focusing on optimizing the maximum completion time, total machine load, and maximum machine load. They proposed a non-dominated sorting teaching-learning-based optimization algorithm (NSTLBO) to minimize the energy consumption and makespan of all jobs.

However, in practice, many resources are not like human resources, as they require installation, setup, or relocation before use, which takes a significant amount of time. This preparation process is known as the setup time, which is crucial in production scheduling and may significantly impact it. Setup time comprises two types [19]: sequence-related and independent. The former depends on the sequence, with each process requiring resource adjustment beforehand. Wu et al. [20] delved into the dual-resource-constrained flexible job shop scheduling problem, considering fixture loading and unloading time. They proposed a

similarity-based scheduling algorithm, SSA4STR, and utilized an improved non-dominated sorting genetic algorithm, NSGA-II, to overcome the limitation of previous approaches that only consider machine tool resources. Cawley et al. [21] employed a winery as a case study and devised a decomposition algorithm tailored to different types of settings. This algorithm effectively addresses the batch sizing and scheduling problems associated with bottling lines featuring parallel line sequences and interrelated setup times. Zhao et al. [22] investigated a distributed blocked flow shop sequence-related scheduling problem with production efficiency measures, jointly considering manufacturing span, total tardiness, and total energy consumption. They proposed a cooperative whale optimization algorithm (CWOA). Conversely, the latter type involves the sequence's setup time independent, which only relates to the current processing operation. Qiao et al. [23] investigated a two-stage hybrid flow shop scheduling problem, where the first stage had a setup time due to the need for machine adjustment. They designed an adaptive genetic algorithm to solve the problem. Sharma et al. [24] developed a mixed integer programming model for solving the dual-criteria flow shop scheduling problem with setup times independent of the sequence, proposing an improved heuristic algorithm based on the desert sparrow optimization (DSO) algorithm. Belabid et al. [25] explored the permutation flow shop problem with independent setup time, where setup time depends on machine type and job preparation method.

In conclusion, the current research on multi-resource scheduling problems always overlooks the significance of mold resources, which directly impact the maximum makespan in various industries, such as manufacturing. While researchers have primarily concentrated on machine energy consumption or artificial skill level, they have overlooked the impact of resource constraints similar to molds, resulting in an inadequate reflection of the actual production scenario. Furthermore, the literature on setup time considerations lacks specific research focusing on optimizing sequence-dependent setup times.

III. FJSP-MRST PROBLEM DESCRIPTION

The flexible job shop scheduling problem with mold resource setup time optimization can be described as a scenario where J jobs must be processed, each comprising of J_i operations, using A machines and M molds. The problem assumes that each procedure must be performed in sequence, with only one machine and one mold available for processing at a time. A mold change is required whenever a different mold is used, with the resulting setup time being determined by the mold used and independent of the machine it is loaded on.

The problem involves two sub-problems:

- 1) Determine the optimal allocation of machines and molds for each operation.
- 2) Determine the optimal processing sequence of each machine to minimize the maximum makespan and setup time.

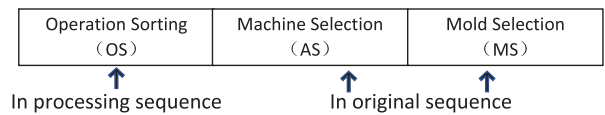


FIGURE 1. Chromosome composition programme.

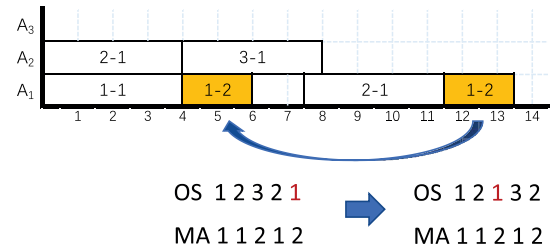


FIGURE 2. Operation insertion method Gantt chart.

An example of the problem is provided in Table.(1), Table.(2), and Table.(3), which provide job information, operation-machine processing time information, and mold change time information. To minimize the makespan and setup time, the solution to the problem is represented using a modified chromosome composition scheme [26] as shown in Fig. 1. In this scheme, each gene of the operation sorting part (OS) is directly encoded with the job number and arranged in the processing sequence. Additionally, the sequence of the machine selection part (AS) and the mold selection part (MS) is also arranged according to the job and job operation sequence. Table.(4) provides a decoding scheme for the problem example in Table.(1), Table.(2), and Table.(3).

In real-world production management, minimizing the makespan is crucial to ensuring the timely delivery of orders. While setup time is already included in makespan, organizing mold changes in a reasonable sequence can significantly enhance enterprise efficiency and reduce workload [27]. Therefore, the objective of FJSP-MRST is to minimize both makespan and setup time, with the former being of greater importance.

IV. METHODS

A. MOLD-MACHINE DOUBLE FUSION INSERTION METHOD

In traditional FJSP, an insertion decoding strategy is commonly employed to adjust randomly generated chromosomes during decoding, aiming to accelerate their convergence speed [28]. This approach helps prevent poor convergence resulting from low-quality randomly generated chromosomes. The operation insertion method [29] is the most frequently used technique, which effectively ensures chromosome quality by adjusting the sequence of operations and placing all operations in their best feasible locations. Fig. 2 illustrates the Gantt chart and its encoding obtained by using the operation insertion method to adjust the existing chromosomes in the traditional FJSP. Notably, the adjusted scheme utilizes more idle time than the original scheme.

TABLE 1. The information about jobs.

	Job1				Job2				Job3			
	O11	O12	O13	O14	O21	O22	O23	O24	O31	O32	O33	O34
Machine	1,2	2,3,4	4	1,3	1,4	1,2,3	3	2,4	2,3	3,4	1	1,2,3,4
Mold	1,3	2,4	1	2,3,4	2,3	4	1,3,4	1,2,3,4	2,3	2,4	1,4	1,2,3

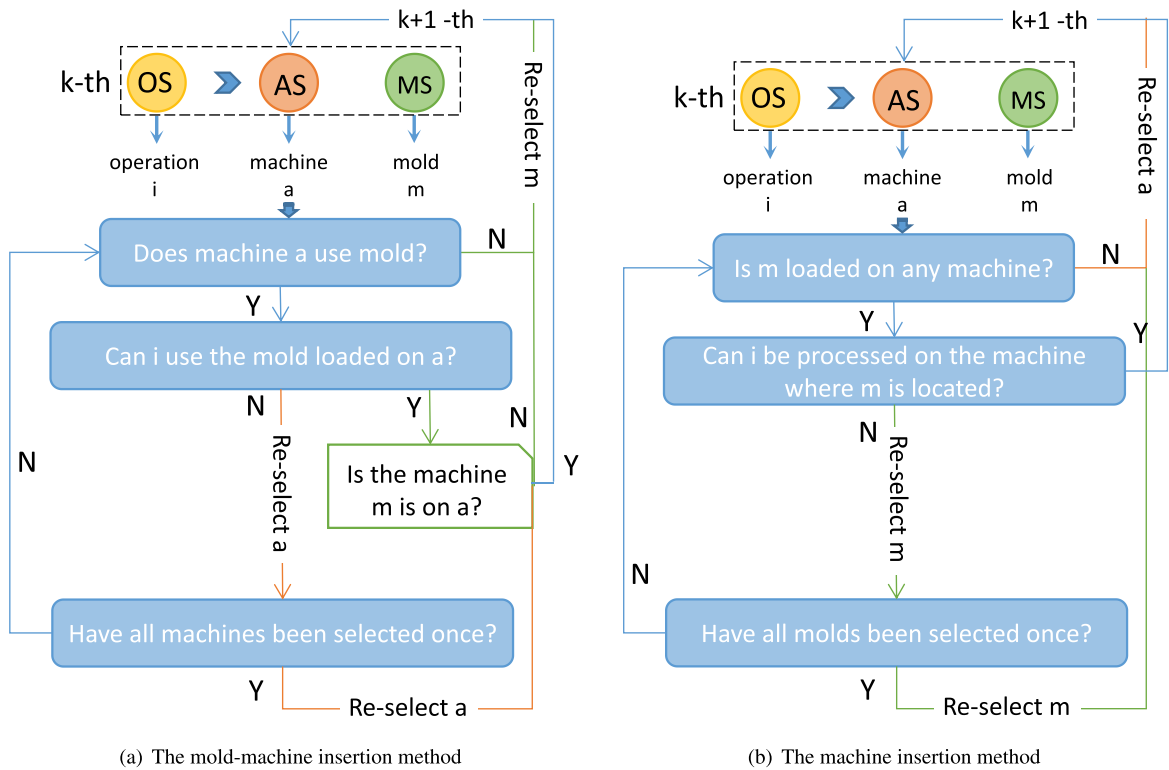


FIGURE 3. The two methods judgment process.

TABLE 2. Machining time for each operation on machines.

	A1	A2	A3	A4
O11	3	2	/	/
O12	/	4	3	5
O13	/	/	/	2
O14	2	/	3	/
O21	1	/	/	1
O22	5	3	5	/
O23	/	/	4	/
O24	/	2	/	2
O31	/	1	2	/
O32	/	/	3	4
O33	4	/	/	/
O34	2	3	4	3

This prompted us to tackle the initial challenge of FJSP-MRST, which is to rationally arrange the process sequence, mold and machine selection by adjusting the

TABLE 3. Setup time for each mold.

M1	M2	M3	M4
0.5	2	1.5	1

existing chromosomes. However, relying solely on the process insertion method to adjust the encoding with mold constraints often leads to suboptimal outcomes, including reverse optimization, wherein only process optimization is considered, resulting in the addition of more mold change conflicts. This occurs because the process insertion method aims to minimize the idle waiting time of the machining process, and the effectiveness of the adjustments varies based on the optimization objectives. Considering that the primary optimization goal of the problem is setup time, we devised an adjustment method to minimize the number of mold changes.

Consequently, we developed a Mold-Machine Insertion Method based on mold change and a Machine Insertion Method based on machine change. These two approaches were integrated to create the Mold-Machine Double Fusion

Algorithm 1 Mold-Machine Insertion Method**Require:** Chromosome with process, mold and machine codes**Ensure:** Updated chromosome

```

1:  $k = 1$ ;
2: while  $k \leq \text{length of process code}$  do
3:   Determine the actual process  $i$  corresponding to the
    $k$ th process code, and machine code  $a$ , mold code  $m$ ;
4:   if machine  $a$  uses mold then
5:     if machine  $a$ 's last used mold  $m$  can be used for
     current process then
6:       Continue to next process ( $k = k + 1$ );
7:     else
8:       Select a new machine gene;
9:       if all genes have been selected once then
10:        Select the machine code with the best
        completion time as the machine gene;
11:       end if
12:       Select a new mold gene;
13:        $k = k + 1$ ;
14:     end if
15:   end if
16: end while

```

Insertion Method, aimed at adjusting the chromosomes and optimizing the setup time by reducing the number of mold changes.

The Mold-Machine Insertion Method is mold-oriented, as illustrated in Fig.3. Algorithm 1 outlines the specific steps involved in this method. The genetic code for the machine and mold corresponding to the current process is obtained, and priority is given to using the mold already loaded on the machine. If no mold exists on the machine, the mold with the lowest usage frequency is selected, and the mold genetic code is modified accordingly. If the mold loaded on the machine cannot be used, the machine with the earliest machining end time is reselected and reevaluated. To prevent algorithmic loops, we have included a check to determine whether the available machine group has been selected exhaustively. Moreover, we prevent repeated mold changes caused by the idle time of the machine by verifying whether another machine uses the last mold used by the selected machine. To avoid reducing the mold or machine utilization rate due to the algorithm always selecting the same machine or mold for different operations, a probability factor is added each time the genetic code is reselected.

In contrast, the Machine Insertion Method is specifically designed for machine mold change, whereby priority is given to the machine that houses the mold for processing. A process is only carried out on the available machine where the mold is located. If such a machine is unavailable, an alternative machine is used. The algorithmic process for this method is depicted in Fig. 3(b), and the machine and mold genes are chosen similarly to the mold-machine insertion method.

Algorithm 2 Machine Insertion Method**Require:** Chromosome with process, machine, and mold codes**Ensure:** Updated chromosome

```

1: Input chromosome (including process, machine, and
   mold codes);
2:  $k = 1$ ;
3: while  $k \leq \text{length of process code}$  do
4:   Determine the actual process  $i$  corresponding to the
    $k$ th process code, and machine code  $a$ , mold code  $m$ ;
5:   if mold  $m$  is loaded on any machine then
6:     if machine  $a$  with the most loaded mold  $m$  can
     process the actual process  $i$  then
7:       Continue to next process ( $k = k + 1$ );
8:     else
9:       Select a new mold gene;
10:      if all mold genes have been selected once
      then
11:        Select the mold with the minimum usage
        as the mold gene;
12:      end if
13:      Select a new machine gene;
14:       $k = k + 1$ ;
15:    end if
16:  end if
17: end while

```

TABLE 4. Decoding solutions.

OS	1	2	2	3	2	1	1	3	3	1	2	3
AS	1	2	4	1	1	2	3	4	2	4	1	3
MS	1	2	1	3	2	4	1	3	2	2	4	1

Algorithm 2 presents the detailed steps involved in this approach.

Due to disparities in biases between the Mold-Machine Insertion Method and the Machine Insertion method, distinct chromosome adjustment effects will be produced, depending on the initial chromosomes under consideration. To illustrate these effects, we employed the Original Operation Insertion Method, as well as the Machine Insertion Method and the Mold Insertion Method customized for the problem, and the Mold-Machine Insertion Method to adjust the original chromosome for the problem examples outlined in Table.(1), Table.(2), Table.(3), following the decoding scheme provided in Table.(4). For the sake of clarity, the probability factor was omitted in the execution steps, and all procedures were executed according to the original plan.

The following is a brief explanation of the other two insertion methods:

The Original Operation Insertion Method involves acquiring the values of the machine and mold genes related to a given process when a new process is decoded. During the machine's idle time, an assessment is made to determine if there is sufficient space to insert the new process. This

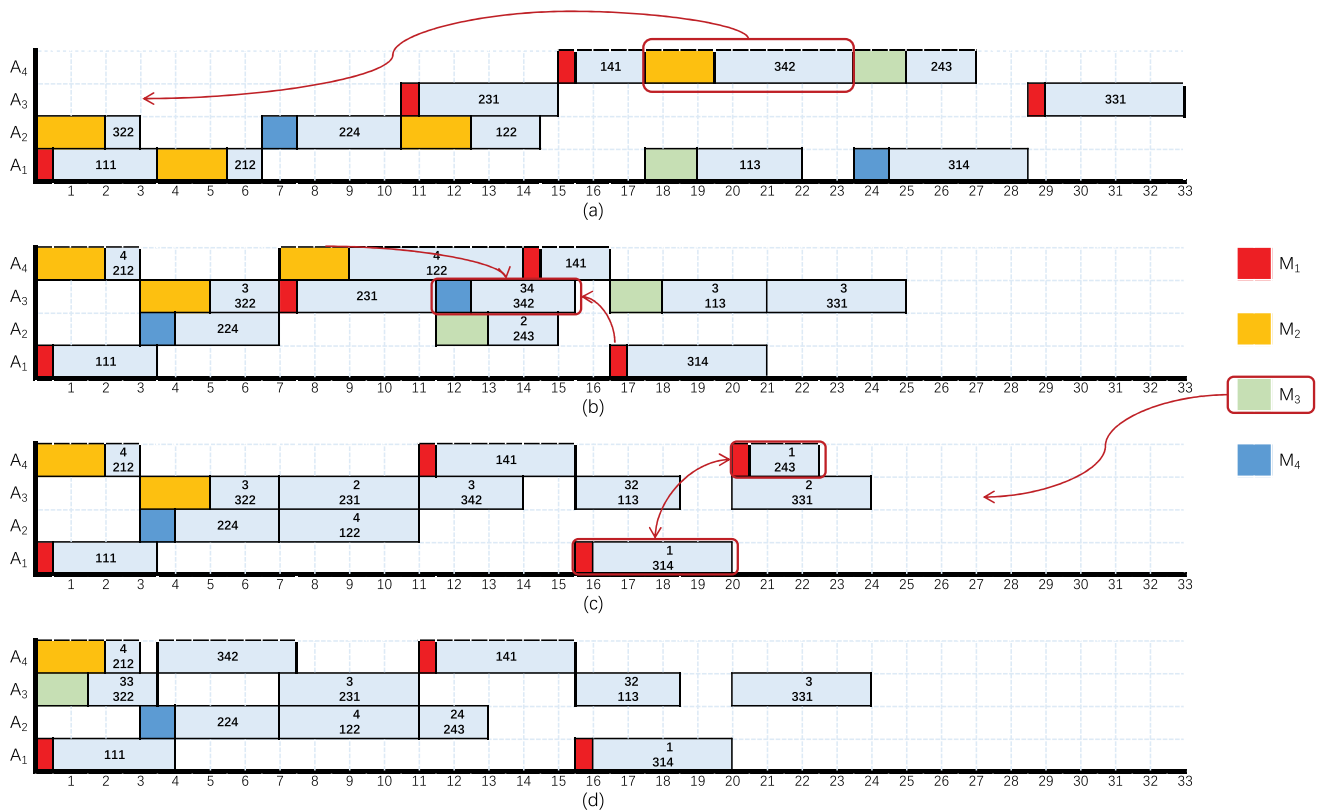


FIGURE 4. Gantt chart of the final effect of chromosome adjustment.

evaluation considers mold usage, completion of the previous process, and the sum of mold change time and processing time. If the idle time is deemed adequate, the process is scheduled to commence at the specific time.

The Mold Insertion Method functions similarly to the mold-machine insertion method. However, it does not verify if the selected mold was used by another machine later. In such cases, the mold number associated with the current process is promptly used to update the mold gene value.

The final Gantt chart in Fig.4 illustrates the impact of adjusting chromosomes using various methods, including operation, machine, mold, and mold-machine insertion. The gene encoding scheme below represents the original chromosome, where the first number corresponds to the job number and the second number represents the second process of that job. The third number represents the machine number, indicating which machine is used for the process. In contrast, the fourth number represents the mold number, showing which mold is used for the procedure. The gene encoding above represents the actual coding after adjustment, with the color-filled portion before the module indicating the mold change time.

In Fig.4(a), the gene with the original code of 342 can be processed using the process insertion method without the influence of the mold. However, when taking mold factors into account, the gene cannot be smoothly inserted in front due to genes 212 and 122 both using the same M_2 mold. This

requires a total setup time and duration of use of 6, making the optimization of the process insertion method less ideal.

In Fig.4(b), the gene with the original code of 342 is adjusted to prioritize the use of machine A_3 . Additionally, the mold is also modified to ensure that the process does not have to wait for gene 122, which uses the same mold, to be released before starting the process. Similarly, the gene with the original code of 331 is adjusted by changing the mold, thereby saving the setup time for M_3 .

Fig.4(c) illustrates that this method dramatically reduces the number of mold changes with machine A_3 . Nonetheless, if the mold is not initially selected, it will be difficult to use it later, leading to a significant reduction in the mold usage rate. Simultaneously, if the last mold used by the selected machine is not checked for use by another machine later, it can result in unnecessary mold changes between machines. For instance, gene 243 changes the mold according to this principle; however, gene 314 occupies M_1 mold, so gene 243 still needs to change the mold.

In Fig.4(d), the leading cause of work intervals is the processing sequence interval, while the interval caused by mold usage is significantly reduced. Upon observing the four adjusted Gantt charts, it becomes evident that, in comparison to mold insertion, the mold-machine insertion method exhibits a more reasonable mold usage. Although the two methods have the same maximum makespan in this example, the mold-machine and machine insertion methods

have significantly enhanced the original chromosome, given their different adjustment methods.

As different guided adjustment methods demonstrate better optimization of a chromosome, this paper proposes a Mold-Machine Double Fusion Insertion Method that combines the Mold-Machine Insertion Method guided by mold change and the Machine Insertion Method guided by machine change to adjust chromosomes. The algorithm's execution process is as follows.

- Step 1:** Insert the original chromosome.
- Step 2:** Use the Mold-Machine Insertion Method to adjust the original chromosome and obtain adjusted chromosome 1.
- Step 3:** Use the Machine Insertion Method to adjust the original chromosome and obtain adjusted chromosome 2.
- Step 4:** Decode adjusted chromosome 1 and adjusted chromosome 2 to obtain setup time 1, maximum makespan 1, setup time 2, and maximum makespan 2, respectively.
- Step 5:** Calculate the fitness of adjusted chromosome 1 and adjusted chromosome 2 based on setup time and maximum makespan, respectively.
- Step 6:** Select the adjusted chromosome with higher fitness and output it.

The fitness calculation in the algorithm is shown in Eq.(1), where TM is the maximum makespan, TC is the setup time, and the weights represent the maximum makespan and mold change time, respectively. Since companies often prioritize maximum makespan in actual production, this paper sets α to 0.65 and β to 0.35. The weight vectors of this formula can be obtained by the Delphi investigation method.

$$Fit = \alpha \cdot TM + \beta \cdot TC \quad (1)$$

B. MOLD-OPERATION LAYER LOADING METHOD

The characteristics of molds and their operations exhibit a strong interdependence, where the initiation of a subsequent operation is contingent upon the completion of the previous procedure, the use of the mold, and the processing of the machine. Consequently, this interdependence results in increased idle waiting time during the execution process, directly impacting the makespan.

Traditionally, we have strictly adhered to this constraint during the decoding and execution process, allowing for the removal of mold changes only when deemed unnecessary. In cases where a mold change was required, we determined the starting time for the current operation of mold change by comparing the ending time of the previous operation, the ending time of the last used mold, and the makespan of the previous job on the machine, selecting the largest time.

However, we have discovered that the process of mold change and operation are not inherently connected. By hierarchically loading molds and operations, hidden situations that can save mold change time can occur. This optimization and

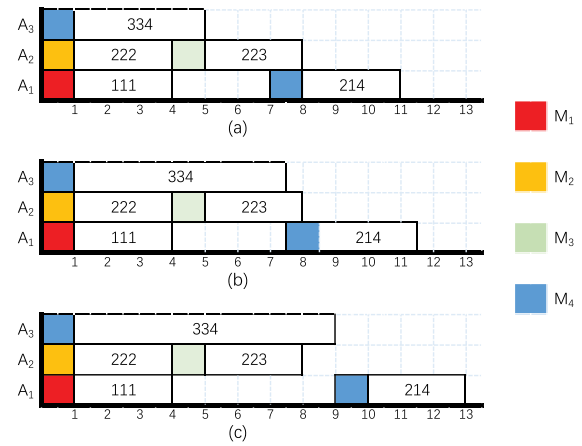


FIGURE 5. Three mold change situations.

compression of setup time can directly reduce the maximum makespan.

Fig.5 illustrates three distinct mold change scenarios wherein the time interval between the previous work makespan of the machine and the subsequent work start time is greater than the mold changing time. In situation Fig.5(a), marked reductions in setup time are observed by employing a hierarchical loading method that prioritizes mold change before processing. As a result, a mold change required for gene number 214 is shifted from time 8 to time 7, resulting in a shorter total setting time than the original loading method. Meanwhile, in situation Fig.5(b), the setup time can be partially compressed, with the mold change for gene number 214 occurring during the time interval between the previous work makespan of the machine and the next work subsequent time, resulting in a more minor impact on the maximum makespan than in situation Fig.5(a). Conversely, in situation Fig.5(c), the setup time cannot be compressed, and gene number 214 must wait for M_4 to finish before initiating the mold changing and processing stage.

Therefore, this paper proposes a Mold-Operation Layered Loading Method, aiming to eliminate the rigid constraints between molds and operations through layered loading, thereby streamlining the loading process. Moreover, the encoding scheme is decoded while implicitly removing mold-changing time. By compressing the setup time and capturing the mold changing start and end times, as well as the processing end time for each operation, the method records the maximum makespan. Fig.6 depicts a decision tree that assesses whether mold changing time can be disregarded and determines the mold makespan. In this context, f denotes the current job, g represents the current operation within the job, e signifies the mold utilized in the current operation, and n denotes the machine for the current operation. M_e stands for the makespan of mold e before its use in the current operation, while A_n represents the time at which machine n finishes processing before undertaking the current operation, and O_f signifies the time at which the previous process of job f concludes its processing. Furthermore, MT_e denotes the setup

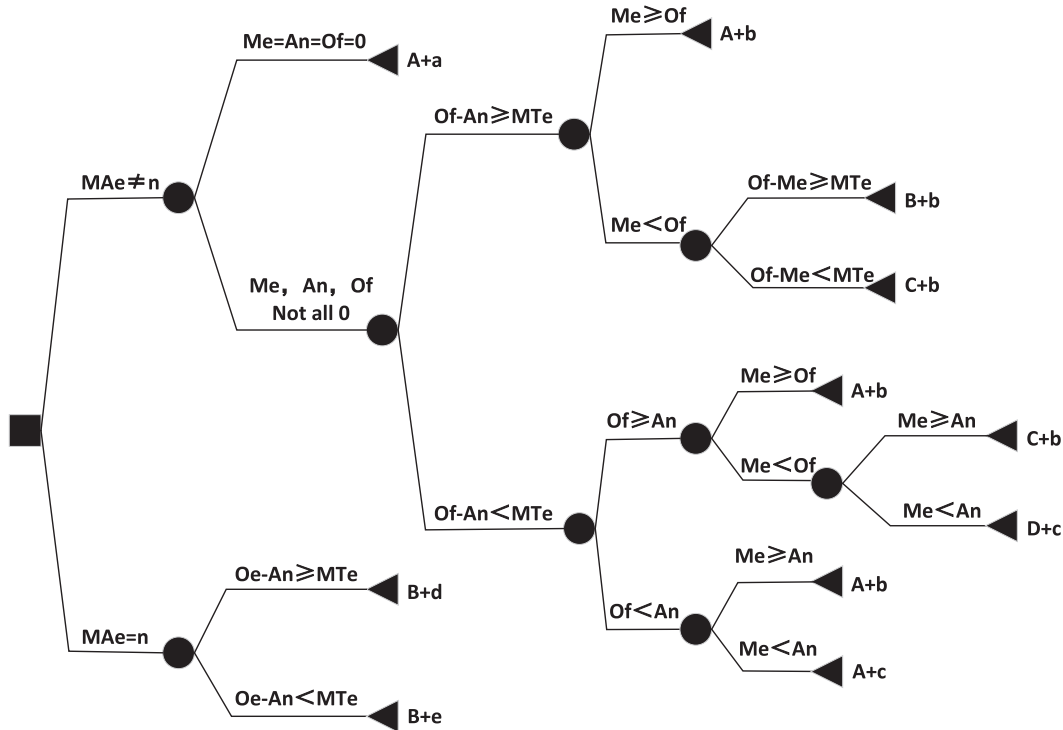


FIGURE 6. The decision tree for the mold-operation layer loading method.

time required by mold e , and AT_{fgn} represents the processing time necessary for job f 's operation g on machine n .

$MA_e = n$ indicates that mold e is currently on the machine to be processed by the current operation, and the mold does not need to change position. Conversely, the mold needs to be changed if they are not equal. When $M_e = 0$ and $A_n = 0$ and $O_f = 0$, both the mold and the machine are idle, and the job has not yet started processing. If none of them is 0, it is necessary to further judge the size of the three and the difference between them after subtracting them pairwise and the setup time. There are four nodes A-D that respectively make decisions about mold changing time in the decision tree, as well as five nodes a-e that respectively decide to update M_e , A_n , and O_f . The details of the resulting nodes are as follows:

- 1) Result A. No setup time can be saved, and the total setup time equals the current total setup time plus MT_e .
- 2) Result B. All setup time can be saved, and the total setup time remains unchanged.
- 3) Result C. Part of the setup time can be saved, and the saved part is the difference between O_f and M_e .
- 4) Result D. Part of the setup time can be saved, and the saved part is the difference between O_f and A_n respectively.
- 5) Result a. M_e , A_n , and O_f are updated to the sum of MT_e and AT_{fgn} , that is, the mold usage time starts from 0.
- 6) Result b. M_e , A_n , and O_f are updated to the sum of MT_e , AT_{fgn} , and M_e , that is, the mold usage time starts from the previous time the mold was finished being used.

- 7) Result c. M_e , A_n , and O_f are updated to the sum of MT_e , AT_{fgn} , and A_n , that is, the mold usage time starts from the previous time the machine was finished being used.
- 8) Result d. M_e , A_n , and O_f are updated to the sum of AT_{fgn} and O_f , that is, the mold usage starts from the time when the previous process of the task ends processing, but the setup time has been omitted.
- 9) Result e. M_e , A_n , and O_f are updated to the sum of AT_{fgn} and A_n , that is, the mold usage time starts from the previous time the machine finished being used, but the setup time has been omitted.

This decision tree efficiently calculates the required setup time. By recording the values of M_e , A_n , and O_f each time, a Gantt chart can be generated to fulfill visualization requirements. Moreover, the Mold-Operation Layer Loading Method circumvents the inclusion of implicit setup time in the overall time progress, facilitates the layered loading of molds and operations, and consequently reduces the maximum makespan.

C. MULTI-OBJECTIVE DIFFERENTIAL EVOLUTIONARY ALGORITHM

The multiple resource-constrained job shop scheduling problem constraints are widely regarded as a typical NP-Hard problem. In tackling this formidable challenge, researchers frequently resort to employing metaheuristic algorithms, including the genetic algorithm [30], [31], grey wolf optimization algorithms [32] and particle swarm algorithms [33],

differential evolution algorithm [34]. Specifically, the FJSP-MRST presents a comprehensive problem that encompasses mold changeover time, machine, and mold resource constraints, thus manifesting as an evident NP-hard problem. Based on the emphasis on the adjustment of machines and molds in the Mold-Machine Double Fusion Insertion Method proposed in this paper, we have chosen to integrate the Differential Evolution algorithm with this method to achieve triple adjustments of operations, molds, and machines, to search for the optimal solution of FJSP-MRST.

Since the Mold-Machine Double Fusion Insertion Method produces large adjustments to the mold and machine codes, which enables the initially generated process codes to obtain more reasonable machine and mold solutions, the traditional differential evolutionary algorithm is adopted in this paper. The specific algorithm flow is shown in Fig.7.

The adjustment scheme includes:

- (1) Focus on performing crossover and mutation on the operation coding to ensure that the offspring have a more efficient operation sequencing scheme. Each time after the next generation is generated, it is adapted using the Mold-Machine Double Fusion Insertion Method to ensure the chromosome quality of each generation;
- (2) The chromosomes were decoded using the Mold-Operation Layer Loading method to remove the invisible mold change time;
- (3) To utilize the optimal chromosome in each generation, 60% of the offspring are generated by using the real number code that corresponds to that chromosome, adding a random number generated from an exponential distribution with a mean value of 3, and then cross-mutating that code;
- (4) In order to maintain genetic diversity in the next generation, the remaining offspring are produced by combining an arbitrarily selected individual from the superior segment of the parent generation population with the differential component of two arbitrarily selected individuals, which is then multiplied by a scaling factor.

The iterative process of each generation of chromosomes is completed through the above operation. Finally, the best chromosome from multiple generations is output to complete the entire operation process of the improved MODE.

V. COMPUTATIONAL EXPERIMENTS AND ANALYSIS

All experiments were performed on a desktop computer equipped with an Intel Core i5-7200U, 2.50GHz CPU, 8.00G RAM, Win10 64 OS, and Matlab. For the improved MODE algorithm, the population size was set to 100, the number of iterations to 10, the mutation coefficient to 0.7, the scaling factor to 0.5, and the crossover coefficient to 0.9 [35]. An adaptive scaling factor was applied, utilizing the formula indicated in Eq.(2). In this formula, G_{max} represents the

TABLE 5. MKm example scenarios.

Instance	Job	Max Operation	Max Machine	Max Mold
MKm-01	10	6	6	6
MKm-02	10	6	6	6
MKm-03	15	10	8	8
MKm-04	15	9	8	8
MKm-05	15	9	4	4
MKm-06	10	15	15	15
MKm-07	20	5	5	5
MKm-08	20	14	10	10
MKm-09	20	14	10	10
MKm-10	20	14	15	15

TABLE 6. MFJSm example scenarios.

Instance	Job	Max Operation	Max Machine	Max Mold
MFJSm-01	5	3	6	6
MFJSm-02	5	3	7	7
MFJSm-03	6	3	7	7
MFJSm-04	7	3	7	7
MFJSm-05	7	3	7	7
MFJSm-06	8	3	7	7
MFJSm-07	9	4	8	8
MFJSm-08	9	4	8	8
MFJSm-09	11	4	8	8
MFJSm-10	12	4	8	8

maximum iteration of the population, while G corresponds to the current iteration.

In this paper, two well-known datasets are selected to evaluate the performance of the proposed algorithm. These datasets contain 30 open problems for FJSP. The first dataset is the MK data from Brandimarte [36], which contains 10 problems. The second dataset is the Fattahi data from Fattahi et al [37], which contains 20 problems. Since these datasets are designed based on FJSP problems, in this paper, the above datasets are expanded for FJSP-MRST by adding data on the total set of molds, the set of molds that can be used for operation in each process, and the mold change time for each mold. Among them, the number of generated molds is the same as the number of machines, the number of molds available for operation in each process obeys the uniform distribution $U[0, M_{max}]$, M_{max} is the maximum number of machines available for operation, and its set of available molds is arbitrarily selected in the set of molds according to the number of generated molds.

In the Brandimarte example, the mold change time for each mold obeys a uniform distribution $U[0, 3]$. In the Fattahi instance, the mold change time for each mold is a random integer between [50, 110]. Since the above two datasets have been transformed in this paper, the Brandimarte dataset will be referred to as the Mkm instance in the latter part

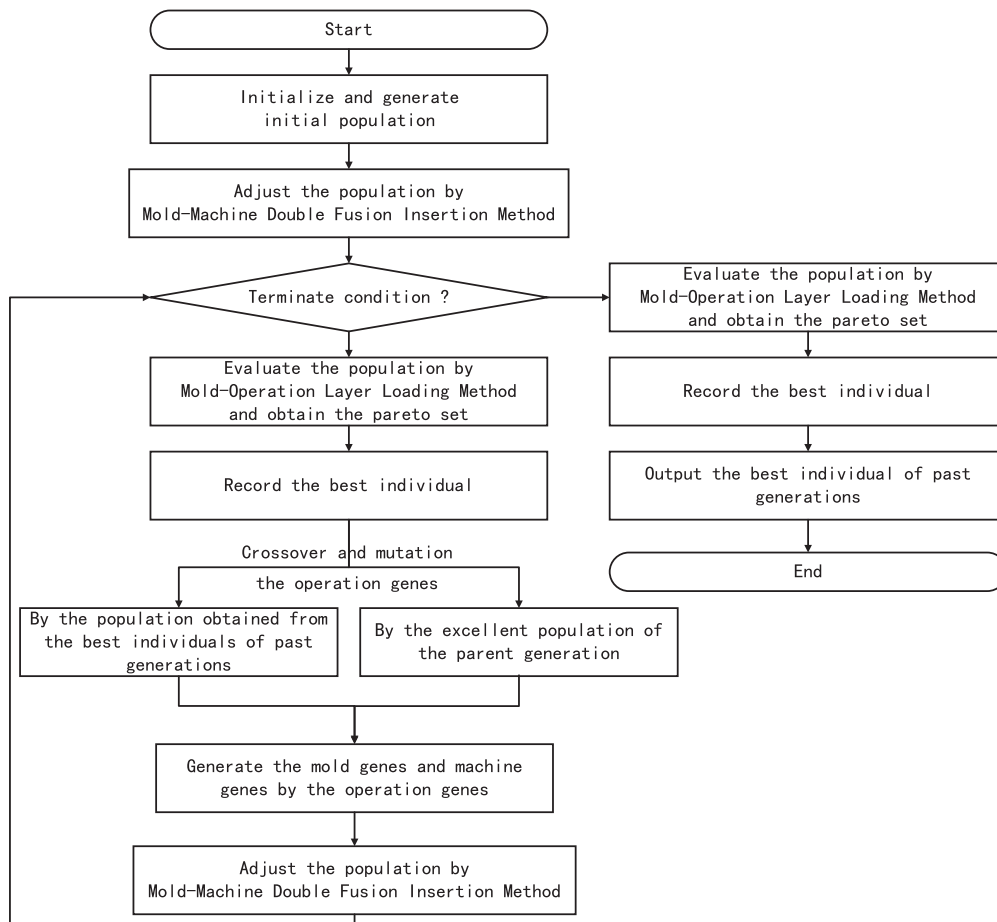


FIGURE 7. Procedural workflow of the improved differential evolution algorithm.

TABLE 7. SFJSm example scenarios.

Instance	Job	Max Operation	Max Machine	Max Mold
SFJSm -01	2	2	2	2
SFJSm -02	2	2	2	2
SFJSm -03	3	2	2	2
SFJSm -04	3	2	2	2
SFJSm -05	3	2	2	2
SFJSm -06	3	3	3	3
SFJSm -07	3	3	5	5
SFJSm -08	3	3	4	4
SFJSm -09	3	3	3	3
SFJSm -10	4	3	5	5

of the paper for ease of discussion, and the instances are listed in Table 5. The first 10 problems in the Fattahi dataset are referred to as the MFJSm algorithm and the examples are listed in Table 6, and the last 10 problems are referred to as the SFJSm algorithm and the examples are listed in Table 7. To evaluate the proposed algorithm and module, we conducted the following numerical experiments:

- (1) Analysis of the Performance of the Mold-Machine Double Fusion Insertion Method;
- (2) Analysis of the Performance of the Mold-Operation Layer Loading Method;
- (3) Analysis of the Performance of the improved MODE.

$$F = 0.8 \cdot [(Gmax - G + 1) \setminus Gmax] \quad (2)$$

A. ANALYSIS OF THE PERFORMANCE OF THE MOLD-MACHINE DOUBLE FUSION INSERTION METHOD

In describing the “Mold-Machine Double Fusion Insertion Method,” we initially conducted a simple comparison of the Operation Insertion Method, Machine Insertion Method, Mold Insertion Method, and Mold-Machine Insertion Method. However, this comparison was limited in representing a broader range of scenarios. Consequently, we designed additional comparative experiments for these methods.

In this experiment, we included a no-adjustment approach as a point of comparison. Additionally, addressing the limitation of the low probability of using a mold in the early stages of the operation, we undertook a redesign

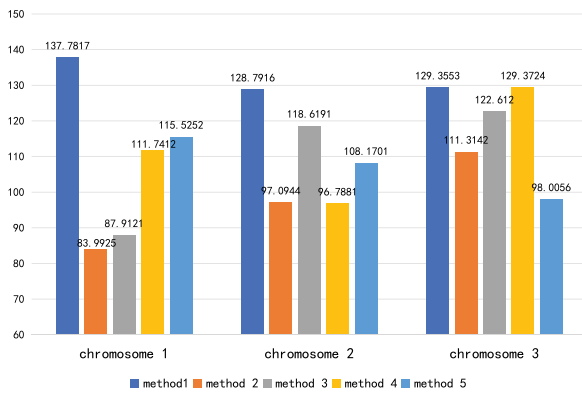


FIGURE 8. Comparison of chromosome adjustments.

of the Mold-Machine Insertion Method. The selection of machines was modified from prioritizing the machine with the earliest completion time to the random selection of available machines. This newly devised approach was termed the mold-machine insertion with random machine method, and we included this method in our comparative analysis.

For ease of reference, we denoted the no-adjustment approach as method1, the Mold-Machine Insertion Method as method2, the Mold Insertion Method as method3, the Mold-Machine Insertion With Random Machine Method as method4, and the Machine Insertion Method as method5. Here, due to the poor performance of the Operation Insertion Method, we excluded experiments related to it.

To illustrate the rationale behind the fusion of the Mold-Machine Double Fusion Insertion Method, which combines method2 and method5, we designed two small experiments. Firstly, we randomly generated three chromosomes and adjusted them using different methods. As shown in Fig. 8, adjusting chromosome 1 using method2 minimized the maximum completion time, yielding the most favorable adjustment. Conversely, for chromosome 3, method5 was the most suitable adjustment method. This indicates that due to the different biases of adjustment methods, each method has the potential to make the initially generated chromosomes more reasonable. Therefore, the superiority of each method cannot be easily judged based on a single case.

Subsequently, we conducted a second experiment, in which we adjusted 500 randomly generated chromosomes for the MKm01 instance using methods 1-5 and tested their adjustment effects. The results are reported in Table (8). It is evident that in most cases, methods 2 and 5 were able to adjust the chromosomes to the optimal level. Thus, it can be inferred that the Mold-Machine Double Fusion Insertion Method, which combines methods 2 and 5, can adjust chromosomes to the optimal state in the vast majority of cases.

In order to test the five methods as well as the Mold-Machine Double Fusion Insertion Method (our algorithm) applied to the adjusted effects post-algorithm. We used the same initial chromosome population and applied different adjustment methods. To mitigate the impact of crossover

and mutation, each algorithm using different adjustment methods was executed 10 times. The results are reported in Table (9), where the first two rows respectively display the best maximum completion time and its corresponding setup time after 10 executions. The third and fourth rows show the variance of the maximum completion time and setup time after 10 executions. A larger variance indicates greater data fluctuation, while a smaller variance signifies reduced data fluctuation. Therefore, a smaller variance demonstrates less fluctuation in the solutions generated by the method, thereby minimizing the random instability of the algorithm results to the greatest extent. The fifth and sixth rows respectively display the average maximum completion time and setup time after 10 executions. For each indicator, the best individual is highlighted in bold. The solutions obtained from executing the algorithm 10 times using different adjustment methods are shown in Fig. 9, providing a visual representation of the data fluctuation. It is evident from the results that our algorithm performed the best.

B. ANALYSIS OF THE PERFORMANCE OF THE MOLD-MACHINE DOUBLE FUSION INSERTION METHOD

To validate the effectiveness of the Mold-Operation Layer Loading Method, we randomly generated chromosomes for the MKm01 dataset and decoded them using both the Mold-Operation Layer Loading Method and the traditional method. To mitigate the impact of varying mold changeover times and setup times for each mold across different chromosomes, we standardized the changeover time for all molds to 2. Fig. 10 displays the maximum completion time obtained after decoding the chromosomes using the two methods, along with their corresponding setup times. The lines between the points represent the differences obtained from using the two decoding methods. Overall, there is a significant difference in the setup times obtained from the two methods, often exceeding 10. Additionally, the indirectly affected maximum completion times also exhibit differences. This suggests that the Mold-Operation Layer Loading Method, by layering the loading of molds and operations, can advance the compressed changeover processes, optimize setup times, reduce maximum completion times, and effectively eliminate hidden changeover times.

C. ANALYSIS OF THE PERFORMANCE OF THE IMPROVED MODE

To evaluate the performance improvement of the improved MODE, we enhanced the NSGA-II (Nondominated Sorting Genetic Algorithm 2), MOEA/D (Multi-objective Evolutionary Algorithm Based on Decomposition), and SMO (Spider Monkey Optimization) algorithms [38] for solving the FJSP problem to adapt them for FJSP-MRST. The enhancements include:

- (1) Addition of mold encoding, where most operations related to mold encoding are similar to machine encoding;

TABLE 8. Effect of chromosome adjustment by different methods.

	Method1	Method2	Method3	Method4	Method5
Makespan	0	193	86	73	148
The setup time	0	143	27	18	312
Fitness	0	210	77	51	162

TABLE 9. Effect of application of different adjustment schemes.

	Method1	Method2	Method3	Method4	Method5	Our algorithm
Makespan	89.3538	69.2206	68.1709	74.8849	67.0558	66.5904
The setup time	37.7174	9.5974	22.5891	17.4984	21.0513	19.9006
Var makespan	20.6295	11.2062	20.9553	18.9435	16.0965	10.3300
Var setup time	10.6174	30.0670	7.8331	20.8983	14.6625	9.9257
Mean makespan	100.5673	75.1289	76.1739	80.5221	75.2522	71.1352
Mean setup time	44.3975	19.1187	21.7648	22.8953	18.6094	16.5130

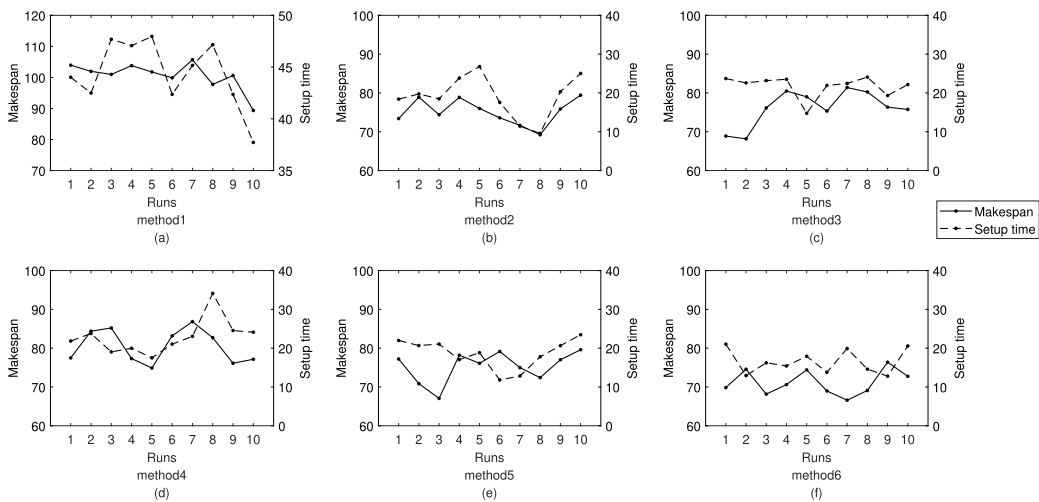


FIGURE 9. Results of the algorithm using different adjustment methods.

TABLE 10. Reference set of MKm.

Instance	Setup time	Makespan
MKm-01	29.2248	82.6975
MKm-02	18.8840	67.2387
MKm-03	87.3825	417.0565
MKm-04	48.2920	139.7003
MKm-05	89.3888	306.4480
MKm-06	21.3902	177.0879
MKm-07	27.9528	348.5843
MKm-08	112.6453	761.8579
MKm-09	192.2571	708.4786
MKm-10	82.2575	613.1021

TABLE 11. Reference set of MFJSm.

Instance	Setup time	Makespan
MFJSm-01	285	801
MFJSm-02	258	792
MFJSm-03	434	1120
MFJSm-04	394	1055
MFJSm-05	718	1304
MFJSm-06	775	1325
MFJSm-07	602	1761
MFJSm-08	893	1905
MFJSm-09	1321	2379
MFJSm-10	1555	2988

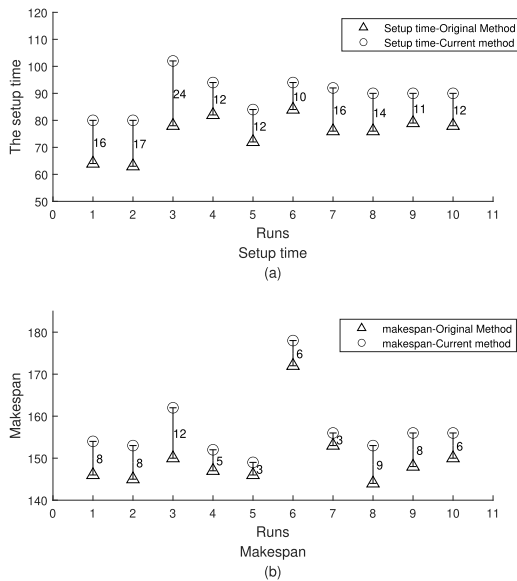
(2) Application of the Mold-Machine Double Fusion Insertion Method to adjust each individual in the initially generated population, aiming to optimize the quality of the initial population;

(3) Decoding operations of the chromosomes using the Mold-Operation Layer Loading Method.

These modifications were made to enable the algorithms to effectively address the FJSP-MRST problem. The improved

TABLE 12. Reference set of SFJSm.

Instance	Setup time	Makespan
SFJSm-01	102	225
SFJSm-02	108	257
SFJSm-03	115	514
SFJSm-04	236	652
SFJSm-05	72	234
SFJSm-06	124	641
SFJSm-07	154	509
SFJSm-08	172	289
SFJSm-09	184	397
SFJSm-10	506	971

**FIGURE 10.** Chromosome decoding results for different scenarios.

algorithms were then compared and evaluated against the original versions to assess their performance in terms of solution quality, convergence, and diversity. The results demonstrated the effectiveness of the enhancements in improving the algorithms' performance in solving the FJSP-MRST problem.

In addition, to further explore the effectiveness of the two methods proposed in this study, we conducted a comparative analysis between the original MODE algorithm and the algorithm proposed in this study. The remaining parts of the algorithm were kept the same as the improved MODE, but excluded the two modules proposed in this study, using a conventional decoding approach.

This comparative analysis aimed to evaluate the impact of the proposed modules on the overall performance of the algorithm, particularly in terms of solution quality and convergence. The results provided insights into the effectiveness of the enhancements introduced in the proposed algorithm, shedding light on their contributions to improving the algorithm's performance in solving the FJSP-MRST problem.

Multi-objective optimization approaches are typically assessed based on various metrics, such as convergence, diversity, and distribution uniformity. To evaluate these metrics, several indicators are commonly utilized, including GD(Generational Distance), IGD(Inverted Generational Distance), Hypervolume [39], Spacing [40], and Spread. However, due to the absence of an appropriate reference set for our specific problem, this study chose to focus on hypervolume, spacing, and IGD as the three primary metrics to evaluate its outcomes.

$$HV = \delta \left(\bigcup_{i=1}^{|S|} v_i \right) \quad (3)$$

The hypervolume (HV) indicator measures the volume of the region in the objective space enclosed by the non-dominated solution set obtained by the algorithm and the reference point. A larger HV value indicates better overall performance of the algorithm. The calculation of HV is based on Eq.(3), where δ represents the Lebesgue measure used for volume measurement, S denotes the number of non-dominated solution sets, v_i represents the hypervolume formed by the reference point and the i -th solution in the solution set. In this study, the reference point selected was obtained using an improved genetic algorithm for the FJSP-MRST. The optimal results were obtained through multiple runs, and the specific values for different instances are presented in Table .(10) to Table .(12).

The spacing metric represents the standard deviation of the minimum distance between each solution and the others within the set. A smaller spacing value indicates that the solutions are more uniformly distributed relative to one another. The calculation of this metric is defined by Eq.(4), in which P denotes the number of Pareto solutions, d_i signifies the Euclidean distance between solution i and its nearest counterpart, and $\bar{d}$ denotes the average value of d_i .

The Inverted Generational Distance (IGD) is a commonly used metric that simultaneously evaluates the convergence and diversity of an algorithm. It measures the distance between the solution set generated by the algorithm and the true Pareto front. A smaller IGD indicates a more optimal solution set generated by the algorithm. Its calculation is defined by Eq.(5), where A represents the obtained true solution set, Z represents the Pareto front solution set, and $\bar{d}_i$ denotes the Euclidean distance from Z_i to the closest reference point in A . Obtaining the Pareto front solution set can be challenging; hence, this study will use non-dominated sorting to rank the solutions obtained by all the algorithms used, including NSGA, MOEA/D, SMO, and our proposed algorithm. The top ten ranked solutions will constitute the Pareto front solution set.

In addition, we have also introduced an additional metric called "Fit," which is calculated using Eq.(1). As the FJSP-MRST problem is motivated by real-world factory scenarios, we believe it is important to include a metric like Fit that can reflect the perspectives of the factory. A smaller Fit value indicates that the algorithm has achieved better setup time and makespan results.

TABLE 13. Experimental results of MKm.

		MKm-01	MKm-02	MKm-03	MKm-04	MKm-05	MKm-06	MKm-07	MKm-08	MKm-09	MKm-10
Min(makespan)	MODE	130.709	108.8981	670.9543	227.0018	468.9073	322.7014	498.5853	1021.0474	1001.9054	763.7168
	Our Algorithm	69.2206	59.5705	364.723	116.9562	290.2447	175.9896	294.7055	733.0733	653.093	526.384
	NSGA-II	71.8027	59.5266	377.128	125.7999	305.6957	174.7426	310.8445	758.1837	677.7211	548.9792
	MOEA/D	64.8327	57.7353	365.8518	127.8873	283.2117	172.7238	281.5149	677.7074	617.4862	485.6632
	SMO	79.9887	64.6504	396.8475	124.3448	297.4866	183.1261	345.4988	790.252	734.011	529.5876
Min(setup time)	MODE	57.9683	50.0863	276.038	118.6266	151.1548	141.3992	93.3999	227.4833	489.0713	299.6846
	Our Algorithm	9.5974	3.0616	74.9901	33.7041	53.1125	16.7108	33.2561	63.5609	166.8703	76.2214
	NSGA-II	11.105	8.0363	64.1038	36.3957	50.1745	20.4311	21.0502	95.2244	172.8219	81.7449
	MOEA/D	18.8314	28.2836	126.5583	41.2216	90.3952	30.8668	58.7735	129.9795	240.5713	144.8435
	SMO	19.8667	8.8998	62.3654	40.3522	60.9501	15.6978	24.3953	100.8715	153.9376	60.2486
Fit	MODE	105.24976	88.31397	532.7336	189.07048	357.69393	259.24563	356.77041	743.29997	822.41347	601.30553
	Our Algorithm	48.35248	39.79239	263.3165	87.81797	207.2484	120.242	203.1982	498.74396	482.9151	368.82709
	NSGA-II	50.558505	41.504995	267.56953	94.50843	216.26328	120.73358	209.4165	526.14795	501.00638	385.4472
	MOEA/D	48.732245	47.427205	282.09908	97.554305	215.72593	123.07385	203.55541	486.0026	485.56599	366.37631
	SMO	58.946	45.13769	279.77877	94.94739	214.69883	124.5262	233.11258	548.96883	530.98531	365.319
Spacing	MODE	3.8723	4.35	22.6449	2.3114	7.8311	5.2529	10.6744	12.2835	23.731	24.776
	Our Algorithm	1.5849	1.3881	4.7348	2.0257	3.1058	2.0817	2.0008	7.8723	15.515	7.3196
	NSGA-II	4.8598	1.0243	5.8243	13.6542	3.5868	3.9378	9.4561	11.3973	16.4752	35.7057
	MOEA/D	5.5051	3.3234	20.3033	4.4318	7.3418	6.8647	15.7964	50.4392	38.6618	37.7482
	SMO	4.3272	1.1846	5.2386	2.9966	11.8997	3.4346	17.11	23.2003	18.8479	12.3012
HV	MODE	0	0	0	0	0	0	0	0	0	0
	Our Algorithm	0.1677	0.1659	0.0985	0.0978	0.0776	0.046	0.0566	0.061	0.057	0.0668
	NSGA-II	0.1437	0.133	0.1108	0.0981	0.0645	0.0604	0.069	0.0386	0.0524	0.0614
	MOEA/D	0.119	0.0119	0.0155	0.0383	0.0365	0.0007	0.0097	0.0261	0.0006	0.0105
	SMO	0.0583	0.0754	0.0647	0.063	0.0544	0.0199	0.0218	0.0122	0.0165	0.0862
IGD	MODE	59.9324	75.1607	345.3745	148.2973	217.386	192.2433	243.188	229.474	394.4566	303.6827
	Our Algorithm	9.7853	2.8544	26.9759	4.1735	6.958	6.62	5.3905	39.0716	18.8309	11.9911
	NSGA-II	8.0504	8.4673	39.781	6.9426	19.9845	9.8664	10.6796	66.8033	45.2955	6.7847
	MOEA/D	9.4011	10.0552	48.3167	14.1277	31.1896	17.3778	34.4943	16.2872	52.2475	16.6717
	SMO	10.467	18.9459	35.4343	23.6308	32.1276	17.6044	42.0891	68.7751	42.7927	11.8773

Table.(13) to Table.(15) present the experimental results obtained for the MKm, MFJSm, and SFJSm instances of the FJSP-MRST problem using different algorithms. For each instance, the optimal solution is highlighted in bold. A comparison with the original MODE clearly demonstrates the significant improvements achieved by the two methods proposed in our study. These enhancements have resulted in a reduction of setup time and makespan by more than half, thereby validating the effectiveness of our proposed modules. Furthermore, when compared with other algorithms, our algorithm consistently outperforms in terms of the number of superior solutions obtained. In most instances, our algorithm excels in Fit, Spacing, HV, and IGD metrics, indicating its superior performance in achieving optimal setup time and makespan, as well as convergence, solution distribution, and diversity compared to other algorithms.

Fig.11 presents a comparative distribution of Pareto solutions obtained by our algorithm, NSGA-II, MOEA/D, and SMO for the MKm instance. In the figure, “*” denotes the results obtained by our algorithm. From Fig.11, it is evident that the Pareto solution set obtained by our algorithm is more evenly distributed, effectively balancing considerations for setup time and makespan. This indicates that our algorithm generally outperforms other algorithms while maintaining

better stability.

$$Spacing(P) = \sqrt{\frac{1}{|P| - 1} \sum_{i=1}^{|P|} (\bar{d} - d_i)^2} \quad (4)$$

$$IGD(A) = \frac{1}{|Z|} (\sum_{i=1}^{|Z|} \check{d}_i^p)^{1/p} \quad (5)$$

D. DISCUSSIONS

This paper proposes a Mold-Machine Fusion Insertion Method to enhance the efficiency and quality of the improved MODE algorithm by adjusting the generated chromosomes to achieve a more reasonable mold and machine arrangement. To validate the superiority of our approach, we compare two methods, the Mold-Machine Insertion Method and the Machine Insertion Method, with other methods for adjusting randomly generated chromosomes. We also verify the effectiveness of the improved MODE algorithm using different adjustment methods. The Mold-Machine Fusion Insertion Method can optimize most chromosomes to the optimal state, resulting in better quality populations and performance of the improved MODE algorithm. Furthermore, this method effectively reduces the fluctuation of the algorithm's solutions. Moreover, we introduce the Mold-Operation Layered Loading Method and compare it with the traditional method of removing only the unnecessary mold change processes

TABLE 14. Experimental results of MJFSm.

		MFJSm-01	MFJSm-02	MFJSm-03	MFJSm-04	MFJSm-05	MFJSm-06	MFJSm-07	MFJSm-08	MFJSm-09	MFJSm-10
Min(makespan)	MODE	1318	1261	1841	2010	2056	2179	2980	3081	3870	4578
	Our Algorithm	383	542	396	468	656	983	770	1167	1293	1020
	NSGA-II	637	610	786	824	965	1058	1590	1463	1891	2202
	MOEA/D	743	678	883	1296	1076	1277	1682	1720	2224	2561
	SMO	676	671	901	979	994	1277	1665	1670	2136	2536
Min(setup time)	MODE	833	1391	1288	1579	1808	2565	2069	2284	3289	4194
	Our Algorithm	218	186	245	466	183	425	297	623	443	583
	NSGA-II	231	258	212	231	268	458	362	270	1126	766
	MOEA/D	330	258	326	155	372	782	557	683	1255	1381
	SMO	200	258	293	293	386	572	437	458	769	589
Fit	MODE	1148.25	1306.5	1647.45	1859.15	1969.2	2314.1	2661.15	2802.05	3666.65	4443.6
	Our Algorithm	325.25	417.4	343.15	467.3	490.45	787.7	604.45	976.6	995.5	867.05
	NSGA-II	494.9	486.8	585.1	616.45	721.05	848	1160.2	1045.45	1623.25	1699.4
	MOEA/D	598.45	531	688.05	896.65	829.6	1103.75	1288.25	1357.05	1884.85	2148
	SMO	509.4	526.45	688.2	738.9	781.2	1030.25	1235.2	1245.8	1657.55	1854.55
Spacing	MODE	63.6148	113.3292	195.6969	133.4785	107.3483	128.4795	354.4342	208.8044	71.6151	297.9445
	Our Algorithm	24.6901	35.6553	26.5107	48.545	34.4049	109.0598	122.0137	41.6445	93.0062	125.3701
	NSGA-II	30.6958	58.8811	41.1965	56.4113	38.6358	136.4563	142.5514	96.1894	316.1413	163.3858
	MOEA/D	47.9223	36.5515	29.9807	94.5086	43.4615	195.8198	141.2814	150.5857	101.9976	147.3955
	SMO	51.3295	34.2338	60.4245	92.9588	100.9434	121.2679	154.7493	50.5328	102.7818	116.1754
HV	MODE	0	0	0	0	0	0	0	0	0	0
	Our Algorithm	0.1826	0.1382	0.3493	0.0249	0.4169	0.1633	0.3323	0.2176	0.3515	0.4546
	NSGA-II	0.1086	0.0547	0.2213	0.1862	0.2431	0.1579	0.0858	0.2189	0.1078	0.2022
	MOEA/D	0.0132	0.0202	0.0898	0	0.1407	0.0102	0.0209	0.0546	0.0286	0.0706
	SMO	0.0847	0.0209	0.1242	0.0786	0.1862	0.0617	0.0542	0.1311	0.0993	0.1757
IGD	MODE	1113.2667	1549.2996	1680.3054	2189.2449	2558.7726	2587.1026	3820.1506	3224.4918	4450.2209	4370.4141
	Our Algorithm	55.0062	105.6463	101.5215	139.5899	470.7371	153.1821	636.6449	323.3857	892.3253	410.3886
	NSGA-II	94.4088	124.7819	161.9091	172.0875	535.2568	325.2159	1070.8362	350.4535	1235.2243	679.3078
	MOEA/D	173.0389	325.5687	378.7291	472.6592	721.631	579.6775	1475.6616	848.3877	1677.4379	1116.3565
	SMO	102.2901	151.145	238.7285	307.9638	547.7765	257.6353	1109.6022	540.8678	1116.0652	325.1704

TABLE 15. Experimental results of SJFSm.

		SFJSm-01	SFJSm-02	SFJSm-03	SFJSm-04	SFJSm-05	SFJSm-06	SFJSm-07	SFJSm-08	SFJSm-09	SFJSm-10
Min(makespan)	MODE	343	403	437	759	343	845	815	694	690	1258
	Our Algorithm	193	236	362	473	213	298	351	332	184	512
	NSGA-II	241	243	406	608	267	435	469	393	347	825
	MOEA/D	228	274	378	473	205	507	518	393	347	844
	SMO	217	319	362	473	205	624	536	382	362	839
Min(setup time)	MODE	228	232	185	448	332	766	676	546	494	1389
	Our Algorithm	102	108	70	96	72	235	245	154	87	488
	NSGA-II	126	108	70	106	72	235	128	92	97	256
	MOEA/D	192	232	70	236	73	189	183	92	97	292
	SMO	126	127	70	236	73	107	183	155	87	263
Fit	MODE	302.75	343.15	348.8	650.15	339.15	817.35	766.35	642.2	621.4	1303.85
	Our Algorithm	161.15	191.2	259.8	341.05	163.65	275.95	313.9	269.7	150.05	503.6
	NSGA-II	200.75	195.75	288.4	432.3	198.75	365	349.65	287.65	259.5	625.85
	MOEA/D	215.4	259.3	270.2	390.05	158.8	395.7	400.75	287.65	259.5	650.8
	SMO	185.15	251.8	259.8	390.05	158.8	443.05	412.45	302.55	265.75	637.4
Spacing	MODE	48.4497	5.9489	52.9024	70.5991	65.3487	95.4861	132.2846	96.924	68.6605	141.6788
	Our Algorithm	10.067	2.2136	15.5492	30.6529	2.2098	40.7792	42.3138	32.6292	20.2433	23.3128
	NSGA-II	37.3333	1.2368	23.3621	32.7853	43.7631	66.9196	83.0772	33.7895	38.6988	26.9477
	MOEA/D	12.324	2.2136	21.9487	43.8356	3.093	41.574	48.1651	25.9752	41.9926	73.7021
	SMO	13.914	47.4342	39.838	45.5368	9.3832	46.3652	39.9755	36.8728	44.0768	62.7839
HV	MODE	0	0	0	0	0	0	0	0	0	0
	Our Algorithm	0.02	0.015	0.1607	0.1315	0.0157	0.0248	0.0468	0	0.3299	0.2166
	NSGA-II	0	0.0128	0.1259	0.0978	0	0	0	0	0.1237	0.1333
	MOEA/D	0	0	0.148	0.1072	0.0181	0	0.0397	0	0.107	0.1039
	SMO	0.0039	0	0.1607	0.1397	0.0181	0	0	0	0.1069	0.1422
IGD	MODE	306.5671	273.4991	462.3089	501.7755	434.3374	1009.4981	709.8423	873.9443	952.7654	1360.9932
	Our Algorithm	3.7	0.7	8	6.3	6.623	25.632	52.4554	37.961	131.2997	78.6698
	NSGA-II	5.8084	26.32	13.5509	17.3276	38.6534	36.1333	66.4703	45.142	179.7607	96.8067
	MOEA/D	5	86.083	3.8	4.2	7.9	53.3266	141.0076	116.796	214.5869	167.3594
	SMO	17.5823	85.5423	12.4104	18.9	4.6203	55.645	107.7597	55.0018	194.4907	197.7193

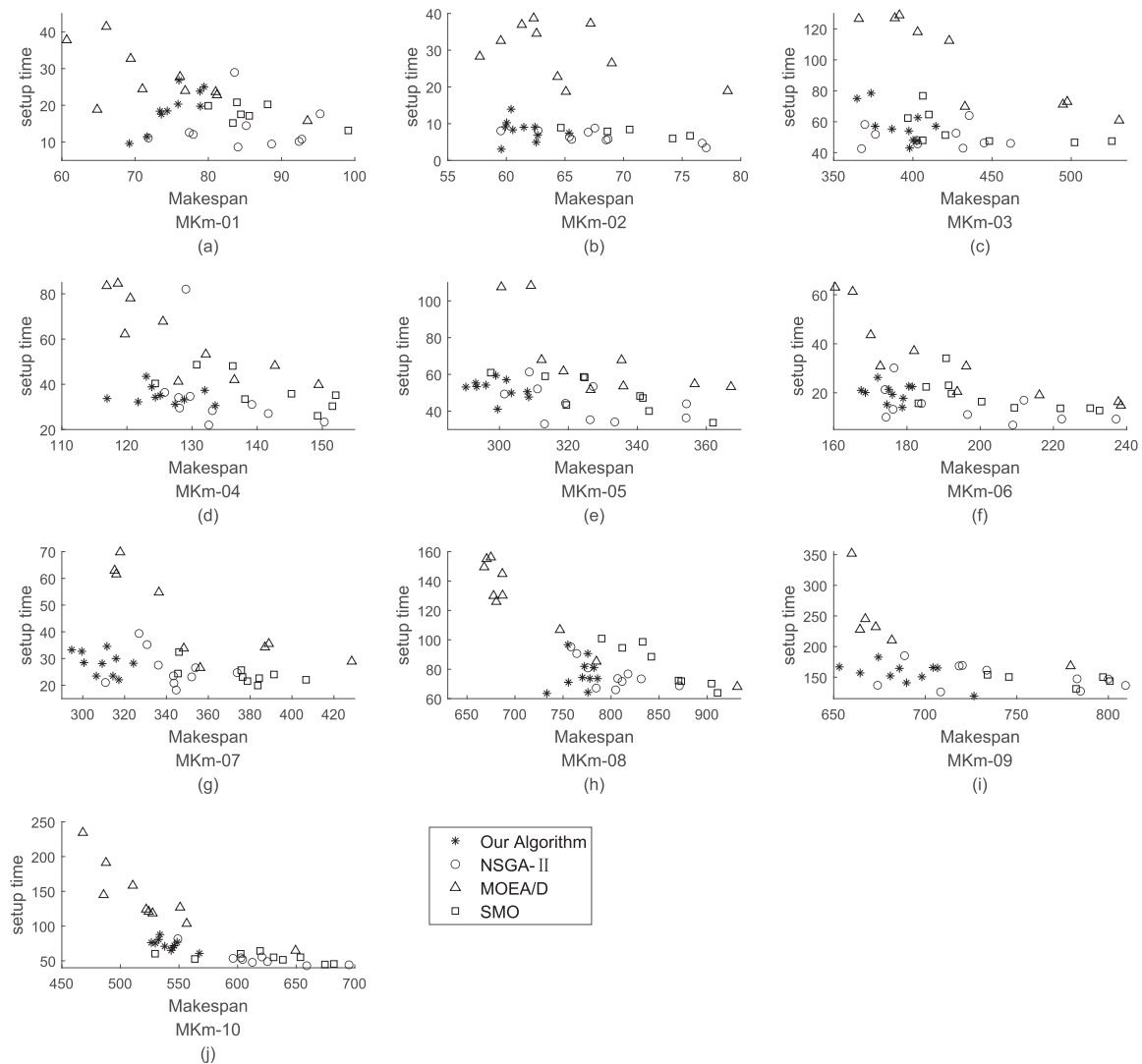


FIGURE 11. Chromosome decoding results for different scenarios.

to decode the chromosomes. Our results demonstrate that the Mold-Operation Layered Loading Method can compress many mold change processes in the chromosome solution, thereby advancing the makespan of the job process and shortening the maximum makespan of all jobs. This method ensures that jobs are completed more efficiently while maintaining the process and arrangement of machines and molds.

In order to validate the performance of the improved MODE algorithm in addressing this problem, an experiment is devised to compare the effectiveness of MODE, NSGA-II, MOEA/D, and SMO, considering their makespan, setup time, fit, HV, Spacing, and IDG. The findings indicate that the improved MODE algorithm frequently yields the optimal solution for the problem and surpasses other algorithms in terms of convergence, distribution, and diversity. The module developed in this study adeptly adjusts the machine and mold encoding. Furthermore, the improved MODE algorithm efficiently modifies the process encoding through crossover

and mutation, thereby rendering the process sequence more rational. The quality of the chromosome is assured through the module's adjustment and the algorithm's refinement.

The exceptional performance of our algorithm can be attributed to its specialized approach in the crossover and mutation processes, which concentrate solely on the process encoding. Additionally, the machine and mold encoding rely entirely on the mold-machine fusion insertion method to generate adjustments. Conversely, the other algorithms primarily acquire the three-segment encoding via their independent evolutionary processes. Consequently, the improved MODE algorithm can attain improved solutions in the same number of iterations. This further highlights the effectiveness of the Mold-Machine Fusion Insertion Method.

VI. CONCLUSION

In practical development environments, the management of multiple resources is often imperative. Within numerous manufacturing companies, molds serve as critical resources

characterized by sequence-dependent setup times and full occupancy during processing. Consequently, research into scheduling problems with such resource constraints holds significant importance. This paper employs mold resources as a case study and introduces a flexible workshop scheduling problem that focuses on optimizing mold setup time, aiming to minimize the maximum makespan and setup time. To achieve this, we propose the Mold-Machine Fusion Insertion Method, which adjusts machine and mold encoding to rationalize the current machine and mold arrangement, thereby eliminating unnecessary setup time. This adjustment method is integrated into the improved MODE algorithm, aligning with the evolution of process encoding to expedite algorithm convergence while reducing the range of fluctuations in heuristic algorithm solutions. Furthermore, we present the Mold-Operation Layered Loading Method, designed to alleviate strong constraints between molds and processes through layered loading. This approach enables the completion of compressible mold change processes in advance, theoretically optimizing the setup time and directly shortening the maximum makespan in practice, thereby facilitating the chromosome decoding step.

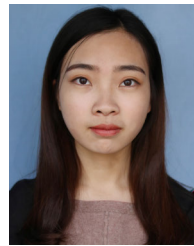
Nevertheless, this paper solely addresses two types of resources: molds and machines. In reality, actual production involves a broader range of resource constraints, including materials and labor. In our forthcoming research, we will persist in examining these constraints and devising more efficient algorithms to tackle problems with greater effectiveness. Additionally, the dynamic nature of the real development environment frequently gives rise to unforeseen circumstances, such as emergency orders and machine damage, which will remain the focus of our ongoing investigation.

Incorporating a broader range of resource constraints and contingencies into the research will facilitate a more comprehensive approach to production scheduling optimization. This will enhance the applicability of the algorithms to various manufacturing companies, encompassing small-scale custom manufacturing and large-scale mass production. Additionally, as smart manufacturing and Industry 4.0 continue to evolve, the need for real-time scheduling and dynamic adjustment will amplify. Consequently, future research can explore the integration of real-time data and intelligent algorithms to achieve dynamic production scheduling optimization.

REFERENCES

- [1] X. Li, X. Guo, H. Tang, R. Wu, L. Wang, S. Pang, Z. Liu, W. Xu, and X. Li, "Survey of integrated flexible job shop scheduling problems," *Comput. Ind. Eng.*, vol. 174, Dec. 2022, Art. no. 108786, doi: [10.1016/j.cie.2022.108786](https://doi.org/10.1016/j.cie.2022.108786).
- [2] C. Destouet, H. Tlahig, B. Bettayeb, and B. Mazari, "Flexible job shop scheduling problem under industry 5.0: A survey on human reintegration, environmental consideration and resilience improvement," *J. Manuf. Syst.*, vol. 67, pp. 155–173, Apr. 2023, doi: [10.1016/j.jmsy.2023.01.004](https://doi.org/10.1016/j.jmsy.2023.01.004).
- [3] Z. Shao, D. Pi, and W. Shao, "A novel discrete water wave optimization algorithm for blocking flow-shop scheduling problem with sequence-dependent setup times," *Swarm Evol. Comput.*, vol. 40, pp. 53–75, Jun. 2018, doi: [10.1016/j.swevo.2017.12.005](https://doi.org/10.1016/j.swevo.2017.12.005).
- [4] Y. E. Demirtas, "Unrelated parallel dedicated machine scheduling with sequence dependent setup times: An application in a textile company," *Int. J. Appl. Decis. Sci.*, vol. 15, no. 6, p. 733, 2022, doi: [10.1504/ijads.2022.126104](https://doi.org/10.1504/ijads.2022.126104).
- [5] A. Allahverdi, "The third comprehensive survey on scheduling problems with setup times/costs," *Eur. J. Oper. Res.*, vol. 246, no. 2, pp. 345–378, Oct. 2015, doi: [10.1016/j.ejor.2015.04.004](https://doi.org/10.1016/j.ejor.2015.04.004).
- [6] Q. Luo, Q. Deng, G. Xie, and G. Gong, "A Pareto-based two-stage evolutionary algorithm for flexible job shop scheduling problem with worker cooperation flexibility," *Robot. Computer-Integrated Manuf.*, vol. 82, Aug. 2023, Art. no. 102534, doi: [10.1016/j.rcim.2023.102534](https://doi.org/10.1016/j.rcim.2023.102534).
- [7] S. Mahmud, R. K. Chakraborty, A. Abbasi, and M. J. Ryan, "Swarm intelligent based metaheuristics for a bi-objective flexible job shop integrated supply chain scheduling problems," *Appl. Soft Comput.*, vol. 121, May 2022, Art. no. 108794, doi: [10.1016/j.asoc.2022.108794](https://doi.org/10.1016/j.asoc.2022.108794).
- [8] W. Ren, Y. Yan, Y. Hu, and Y. Guan, "Joint optimisation for dynamic flexible job-shop scheduling problem with transportation time and resource constraints," *Int. J. Prod. Res.*, vol. 60, no. 18, pp. 5675–5696, Sep. 2022, doi: [10.1080/00207543.2021.1968526](https://doi.org/10.1080/00207543.2021.1968526).
- [9] M. Ziaee, J. Mortazavi, and M. Amra, "Flexible job shop scheduling problem considering machine and order acceptance, transportation costs, and setup times," *Soft Comput.*, vol. 26, no. 7, pp. 3527–3543, Apr. 2022, doi: [10.1007/s00500-021-06481-y](https://doi.org/10.1007/s00500-021-06481-y).
- [10] B. Ji, S. Zhang, S. S. Yu, and D. Zhang, "An enhanced adaptive large neighborhood search for unrelated parallel machine scheduling with sequence dependent setup times," *IEEE Access*, vol. 11, pp. 16735–16748, 2023, doi: [10.1109/ACCESS.2023.3245825](https://doi.org/10.1109/ACCESS.2023.3245825).
- [11] M. Balasubramani and R. Uthayakumar, "A sustainable inventory model for setup and ordering cost reduction under a controllable lead time," *Int. J. Math. Oper. Res.*, vol. 1, no. 1, p. 1, 2021, doi: [10.1504/ijmor.2021.10041084](https://doi.org/10.1504/ijmor.2021.10041084).
- [12] A. Allahverdi and H. M. Soroush, "The significance of reducing setup times/setup costs," *Eur. J. Oper. Res.*, vol. 187, no. 3, pp. 978–984, Jun. 2008, doi: [10.1016/j.ejor.2006.09.010](https://doi.org/10.1016/j.ejor.2006.09.010).
- [13] H. Li, X. Wang, and J. Peng, "A hybrid differential evolution algorithm for flexible job shop scheduling with outsourcing operations and job priority constraints," *Expert Syst. Appl.*, vol. 201, Sep. 2022, Art. no. 117182, doi: [10.1016/j.eswa.2022.117182](https://doi.org/10.1016/j.eswa.2022.117182).
- [14] D. Li, J. Wang, R. Qiang, and R. Chiong, "A hybrid differential evolution algorithm for parallel machine scheduling of lace dyeing considering colour families, sequence-dependent setup and machine eligibility," *Int. J. Prod. Res.*, vol. 59, no. 9, pp. 2722–2738, May 2021, doi: [10.1080/00207543.2020.1740341](https://doi.org/10.1080/00207543.2020.1740341).
- [15] Z. He, B. Tang, and F. Luan, "An improved African vulture optimization algorithm for dual-resource constrained multi-objective flexible job shop scheduling problems," *Sensors*, vol. 23, no. 1, p. 90, Dec. 2022, doi: [10.3390/s23010090](https://doi.org/10.3390/s23010090).
- [16] G. Gong, Q. Deng, X. Gong, W. Liu, and Q. Ren, "A new double flexible job-shop scheduling problem integrating processing time, green production, and human factor indicators," *J. Cleaner Prod.*, vol. 174, pp. 560–576, Feb. 2018, doi: [10.1016/j.jclepro.2017.10.188](https://doi.org/10.1016/j.jclepro.2017.10.188).
- [17] J. Wang, K. Zhu, Y. Peng, and K. Zhu, "Energy-efficient scheduling for flexible job shop under multi-resource constraints using non-dominated sorting teaching-learning-based optimization algorithm," *J. Intell. Fuzzy Syst.*, vol. 43, no. 1, pp. 409–423, Jun. 2022, doi: [10.3233/jifs-212258](https://doi.org/10.3233/jifs-212258).
- [18] Z. Liu, X. Liang, L. Hou, D. Yang, and Q. Tong, "Multi-strategy dynamic evolution-based improved MOEA/D algorithm for solving multi-objective fuzzy flexible job shop scheduling problem," *IEEE Access*, vol. 11, pp. 54596–54606, 2023.
- [19] A. Allahverdi, C. T. Ng, T. C. E. Cheng, and M. Y. Kovalyov, "A survey of scheduling problems with setup times or costs," *Eur. J. Oper. Res.*, vol. 187, no. 3, pp. 985–1032, Jun. 2008, doi: [10.1016/j.ejor.2006.06.060](https://doi.org/10.1016/j.ejor.2006.06.060).
- [20] X. Wu, J. Peng, X. Xiao, and S. Wu, "An effective approach for the dual-resource flexible job shop scheduling problem considering loading and unloading," *J. Intell. Manuf.*, vol. 32, no. 3, pp. 707–728, Mar. 2021, doi: [10.1007/s10845-020-01697-5](https://doi.org/10.1007/s10845-020-01697-5).
- [21] A. Mac Cawley, S. Maturana, R. Pascual, and G. L. Tortorella, "Scheduling wine bottling operations with multiple lines and sequence-dependent set-up times: Robust formulation and a decomposition solution approach," *Eur. J. Oper. Res.*, vol. 303, no. 2, pp. 819–839, Dec. 2022, doi: [10.1016/j.ejor.2022.02.054](https://doi.org/10.1016/j.ejor.2022.02.054).

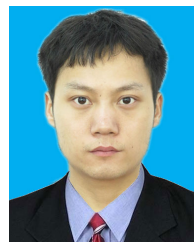
- [22] F. Zhao, Z. Xu, H. Bao, T. Xu, and N. Zhu, "A cooperative whale optimization algorithm for energy-efficient scheduling of the distributed blocking flow-shop with sequence-dependent setup time," *Comput. Ind. Eng.*, vol. 178, Apr. 2023, Art. no. 109082, doi: [10.1016/j.cie.2023.109082](https://doi.org/10.1016/j.cie.2023.109082).
- [23] Y. Qiao, N. Wu, Y. He, Z. Li, and T. Chen, "Adaptive genetic algorithm for two-stage hybrid flow-shop scheduling with sequence-independent setup time and no-interruption requirement," *Expert Syst. Appl.*, vol. 208, Dec. 2022, Art. no. 118068, doi: [10.1016/j.eswa.2022.118068](https://doi.org/10.1016/j.eswa.2022.118068).
- [24] M. Sharma, M. Sharma, and S. Sharma, "Desert sparrow optimization algorithm for the bicriteria flow shop scheduling problem with sequence-independent setup time," *Oper. Res.*, vol. 22, no. 4, pp. 4353–4396, Sep. 2022, doi: [10.1007/s12351-021-00675-w](https://doi.org/10.1007/s12351-021-00675-w).
- [25] J. Belabid, S. Aqil, and K. Allali, "Solving permutation flow shop scheduling problem with sequence-independent setup time," *J. Appl. Math.*, vol. 2020, pp. 1–11, Jan. 2020, doi: [10.1155/2020/7132469](https://doi.org/10.1155/2020/7132469).
- [26] S. C. Kim and P. M. Bobrowski, "Impact of sequence-dependent setup time on job shop scheduling performance," *Int. J. Prod. Res.*, vol. 32, no. 7, pp. 1503–1520, Jul. 1994, doi: [10.1080/00207549408957019](https://doi.org/10.1080/00207549408957019).
- [27] D. Shan and Y. Wang, "A review of the application of genetic algorithms for solving flexible job shop scheduling problems," in *Proc. 2nd Int. Conf. Inf. Technol. Electr. Eng.*, Hunan, China, Dec. 2019, p. 145, doi: [10.1145/3386415.3387092](https://doi.org/10.1145/3386415.3387092).
- [28] H. Cui, X. Li, and L. Gao, "An improved multi-population genetic algorithm with a greedy job insertion inter-factory neighborhood structure for distributed heterogeneous hybrid flow shop scheduling problem," *Expert Syst. Appl.*, vol. 222, Jul. 2023, Art. no. 119805, doi: [10.1016/j.eswa.2023.119805](https://doi.org/10.1016/j.eswa.2023.119805).
- [29] W. Guo, Q. Lei, Y. Song, and X. Lyu, "A learning interactive genetic algorithm based on edge selection encoding for assembly job shop scheduling problem," *Comput. Ind. Eng.*, vol. 159, Sep. 2021, Art. no. 107455, doi: [10.1016/j.cie.2021.107455](https://doi.org/10.1016/j.cie.2021.107455).
- [30] W. Tan, X. Yuan, J. Wang, and X. Zhang, "A fatigue-conscious dual resource constrained flexible job shop scheduling problem by enhanced NSGA-II: An application from casting workshop," *Comput. Ind. Eng.*, vol. 160, Jan. 2021, Art. no. 107557, doi: [10.1016/j.cie.2021.107557](https://doi.org/10.1016/j.cie.2021.107557).
- [31] Z. Liang, M. Liu, P. Zhong, and C. Zhang, "Application research of a new neighbourhood structure with adaptive genetic algorithm for job shop scheduling problem," *Int. J. Prod. Res.*, vol. 61, no. 2, pp. 362–381, Jan. 2023, doi: [10.1080/00207543.2021.2007310](https://doi.org/10.1080/00207543.2021.2007310).
- [32] Z. Zhu, X. Zhou, D. Cao, and M. Li, "A shuffled cellular evolutionary grey wolf optimizer for flexible job shop scheduling problem with tree-structure job precedence constraints," *Appl. Soft Comput.*, vol. 125, Aug. 2022, Art. no. 109235, doi: [10.1016/j.asoc.2022.109235](https://doi.org/10.1016/j.asoc.2022.109235).
- [33] D. B. M. M. Fontes, S. M. Homayouni, and J. F. Gonçalves, "A hybrid particle swarm optimization and simulated annealing algorithm for the job shop scheduling problem with transport resources," *Eur. J. Oper. Res.*, vol. 306, no. 3, pp. 1140–1157, May 2023, doi: [10.1016/j.ejor.2022.09.006](https://doi.org/10.1016/j.ejor.2022.09.006).
- [34] G.-G. Wang, D. Gao, and W. Pedrycz, "Solving multiobjective fuzzy job-shop scheduling problem by a hybrid adaptive differential evolution algorithm," *IEEE Trans. Ind. Inform.*, vol. 18, no. 12, pp. 8519–8528, Dec. 2022, doi: [10.1109/TII.2022.3165636](https://doi.org/10.1109/TII.2022.3165636).
- [35] C. Salto, F. Morero, and C. Bermúdez, "Parallelism and hybridization in differential evolution to solve the flexible job shop scheduling problem," *J. Comput. Sci. Technol.*, vol. 20, no. 1, p. e04, May 2020.
- [36] P. Brandimarte, "Routing and scheduling in a flexible job shop by Tabu search," *Ann. Oper. Res.*, vol. 41, no. 3, pp. 157–183, Sep. 1993.
- [37] P. Fattahi, M. Saidi Mehrabad, and F. Jolai, "Mathematical modeling and heuristic approaches to flexible job shop scheduling problems," *J. Intell. Manuf.*, vol. 18, no. 3, pp. 331–342, Jul. 2007.
- [38] K. Gao, Z. Cao, L. Zhang, Z. Chen, Y. Han, and Q. Pan, "A review on swarm intelligence and evolutionary algorithms for solving flexible job shop scheduling problems," *IEEE/CAA J. Autom. Sinica*, vol. 6, no. 4, pp. 904–916, Jul. 2019.
- [39] M. Ciavotta, G. Minella, and R. Ruiz, "Multi-objective sequence dependent setup times permutation flowshop: A new algorithm and a comprehensive study," *Eur. J. Oper. Res.*, vol. 227, no. 2, pp. 301–313, Jun. 2013, doi: [10.1016/j.ejor.2012.12.031](https://doi.org/10.1016/j.ejor.2012.12.031).
- [40] C. Lu, L. Gao, X. Li, and S. Xiao, "A hybrid multi-objective grey wolf optimizer for dynamic scheduling in a real-world welding industry," *Eng. Appl. Artif. Intell.*, vol. 57, pp. 61–79, Jan. 2017, doi: [10.1016/j.engappai.2016.10.013](https://doi.org/10.1016/j.engappai.2016.10.013).



YI CHENG received the B.S. degree in computer science and technology from Luoyang Institute of Science and Technology, Henan, China, in 2021. She is currently pursuing the M.S. degree with Ningbo University, Zhejiang, China. Her research interests include intelligent scheduling and multi-resource workshop scheduling.



ZHIJUN XIE received the Ph.D. degree from the Renmin University of China. He is currently a Visiting Scholar with the University of Victoria, a Postdoctoral Fellow with Tsinghua University, and a Postdoctoral Fellow with Zhejiang University. He is also an Associate Professor with the Faculty of Electrical Engineering and Computer Science, Ningbo University. His research interests include artificial intelligence and blockchain and the Internet of Things.



YU XIN received the B.S. degree in computer science and technology and the M.E. degree in computer application technology from Harbin University of Science and Technology, in 2008 and 2011, respectively, and the Ph.D. degree in computer application technology from Harbin Engineering University, in 2015. He is currently an Associate Professor and the Doctoral Director of Ningbo University. His research interests include image segmentation, image recognition, and big data visualization analysis.



KEWEI CHEN received the B.Sc. degree in industrial automation from Jilin University, in 1984, the M.E. degree in automatic control from Beijing Research Institute of Mechanical and Electronic Technology, Ministry of Mechanical Industry, in 1989, and the Ph.D. degree in intelligent systems science from Tokyo Institute of Technology, Japan, in 2000. He is currently a Professor and the Doctoral Director of Ningbo University, Zhejiang, China. His main research interests include intelligent manufacturing, high-end service robots, and intelligent computing.



ROOZBEH ZAREI received the B.Sc. degree in electrical and electronics engineering from Azad University, Iran, in 2006, the M.E. degree in electrical, electronics and telecommunications engineering from Universiti Teknologi Malaysia (UTM), Malaysia, in 2012, and the Ph.D. degree in computer science from Victoria University, Australia, in 2017. He is currently an Associate Research Fellow with the School of Information Technology, Deakin University, Melbourne, Australia. His current research interests include data stream mining, anomaly detection, pattern recognition, and data analytics (statistics and machine learning).

...